JavaScript Prototypes & Classes - Complete In-Depth Notes

Introduction

This comprehensive guide explains JavaScript's prototype system and classes, revealing how inheritance works behind the scenes and why arrays and functions are considered objects.

Part 1: Understanding Prototypes

1.1 The Mystery: Hidden Methods

Initial Problem:

```
const obj = {
  name: "Rohit",
  age: 38,
  greet: function() {
    console.log("Hello");
  }
};

// These work, but we never defined them!
obj.hasOwnProperty('name');  // true
obj.toString();              // [object Object]
```

Key Questions:

- Where do `hasOwnProperty()` and `toString()` come from?
- We never defined these methods, yet they work
- How does JavaScript make them available?

1.2 Arrays: The Same Mystery

```
const arr = [10, 20, 30];

// We never defined these!
```

```
arr.length;        // 3
arr.sort();        // Works!
arr.slice();       // Works!
arr.filter();      // Works!
```

The Question: Where do all these array methods come from?

Part 2: The Prototype Chain Explained

2.1 Manual Prototype Linking

Example:

```
const obj = {
  name: "Rohit",
  age: 38,
  greet: function() {
    console.log("Hello");
  }
};

const obj2 = {
  account: 30
};

// Initially, obj2.name is undefined
console.log(obj2.name);  // undefined

// Magic happens here:
obj2.__proto__ = obj;

// Now obj2 can access obj's properties!
console.log(obj2.name);  // "Rohit"
```

2.2 How Prototype Search Works

The Search Algorithm:

First Level: JavaScript looks in the object itself

obj2.name → Look in obj2 → Not found

1.

Second Level: Go up to the prototype

obj2.__proto__ → Points to obj → Found "Rohit"!

2.

Visual Representation:

obj2 (account: 30)
↓ __proto__
obj (name: "Rohit", age: 38, greet: function)
↓ __proto__
Object.prototype (hasOwnProperty, toString, etc.)
↓ __proto__
null (end of chain)

2.3 The DRY Principle (Don't Repeat Yourself)

Why Prototypes Exist:

- Avoid duplicating methods across multiple objects
- Save memory by sharing common functionality
- Define once, use everywhere

Part 3: Object.prototype - The Root

3.1 Built-in Prototype

Every object in JavaScript is connected to `Object.prototype`:

```
const obj = {
  name: "Rohit",
  age: 38
};

// Behind the scenes:
```

```
// obj.__proto__ = Object.prototype

// That's why these work:
obj.hasOwnProperty('name');  // from Object.prototype
obj.toString();              // from Object.prototype
obj.valueOf();               // from Object.prototype
```

3.2 Exploring Object.prototype

```
console.log(Object.prototype);
// Shows:
// - hasOwnProperty()
// - toString()
// - valueOf()
// - isPrototypeOf()
// - propertyIsEnumerable()
// ... and more
```

Part 4: Array Prototype Chain

4.1 The Complete Array Chain

```
const arr = [10, 20, 30];
```

Behind the Scenes:

```
arr (10, 20, 30)
  ↓ arr.__proto__
Array.prototype (sort, map, filter, length, etc.)
  ↓ Array.prototype.__proto__
Object.prototype (hasOwnProperty, toString, etc.)
  ↓ Object.prototype.__proto__
null
```

4.2 Why Arrays Are Objects

Demonstration:

```
const arr = [10, 20, 30];
```

```
// arr inherits from Array.prototype
console.log(arr.__proto__ === Array.prototype);  // true

// Array.prototype inherits from Object.prototype
console.log(Array.prototype.__proto__ === Object.prototype);  // true

// Therefore, arrays can use Object methods!
arr.hasOwnProperty('length');  // true
arr.toString();                // "10,20,30"
```

4.3 Method Search in Arrays

Example:

```
const arr = [10, 20, 30];
arr.toString();  // "10,20,30"
```

Search Process:

1. Look in `arr` → Not found
2. Look in `Array.prototype` → Not found
3. Look in `Object.prototype` → Found `toString()`!
4. Execute the method

4.4 Checking Array's Prototype

```
console.log(Array.prototype);
// Shows array methods:
// - concat()
// - filter()
// - find()
// - map()
// - reduce()
// - sort()
// - slice()
// ... and many more
```

Part 5: Function Prototype Chain

5.1 Functions as Objects

```
function ab() {
  // function code
}

// Functions also have prototypes!
console.log(ab.__proto__ === Function.prototype);  // true
```

The Chain:

```
ab (function)
  ↓ ab.__proto__
Function.prototype (call, apply, bind, etc.)
  ↓ Function.prototype.__proto__
Object.prototype (hasOwnProperty, toString, etc.)
  ↓ Object.prototype.__proto__
null
```

5.2 Why Functions Are Objects

Functions inherit from `Object.prototype`, which is why:

- Functions can have properties
- Functions can use methods like `toString()`
- "Everything is an object" in JavaScript

Part 6: The Prototype Chain Navigation

6.1 Moving Up the Chain

Using `__proto__`:

```
const arr = [10, 20, 30];

// Level 1: The array itself
console.log(arr);

// Level 2: Array.prototype
console.log(arr.__proto__);

// Level 3: Object.prototype
```

```
console.log(arr.__proto__.__proto__);

// Level 4: null (end of chain)
console.log(arr.__proto__.__proto__.__proto__);  // null
```

6.2 The Naming Convention

Understanding the Names:

- `Array.prototype` - The prototype object for all arrays
- `Object.prototype` - The prototype object for all objects
- `Function.prototype` - The prototype object for all functions

6.3 Property Lookup Until null

When looking for a property:

```
arr.someRandomMethod();
```

Search Process:

1. Check `arr` → Not found
2. Check `Array.prototype` → Not found
3. Check `Object.prototype` → Not found
4. Check `null` → **Stop! Property doesn't exist**

Part 7: Classes - Syntactic Sugar

7.1 The Problem Classes Solve

Without Classes (Repetitive):

```
const obj1 = {
  name: "Rohit",
  age: 30,
  greet: function() {
    console.log(`Hello ${this.name}`);
  }
};
```

```
const obj2 = {
  name: "Mohit",
  age: 20,
  greet: function() {  // Duplicate!
    console.log(`Hello ${this.name}`);
  }
};

const obj3 = {
  name: "Mohan",
  age: 10,
  greet: function() {  // Duplicate again!
    console.log(`Hello ${this.name}`);
  }
};
```

Problems:

- Code repetition
- Memory waste (same function stored multiple times)
- Hard to maintain
- Not following DRY principle

7.2 Classes: The Solution

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  sayHi() {
    console.log(`Hi ${this.name}`);
  }
}

// Create objects easily
const person1 = new Person("Rohit", 20);
const person2 = new Person("Mohit", 10);
const person3 = new Person("Mohan", 15);
```

Benefits:

- Write once, use many times
- Methods stored once (in prototype)
- Clean, maintainable code
- Memory efficient

Part 8: How Classes Work Behind the Scenes

8.1 Class as Blueprint

What a Class Defines:

- Properties each object will have
- Methods shared across all objects
- Constructor for initialization

8.2 Behind the Scenes: Person.prototype

When you create a class:

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  sayHi() {
    console.log(`Hi ${this.name}`);
  }
}
```

JavaScript Creates:

```
Person.prototype = {
  sayHi: function() { console.log(`Hi ${this.name}`); }
  // Methods go here
}
```

Note: Properties (name, age) are NOT in the prototype!

8.3 The new Keyword Process

```
const person1 = new Person("Rohit", 20);
```

Step-by-Step:

Create empty object:

```
person1 = {};
```

1.

Set prototype:

```
person1.__proto__ = Person.prototype;
```

2.

Call constructor with `this` pointing to new object:

```
this.name = "Rohit";  // person1.name = "Rohit"
this.age = 20;        // person1.age = 20
```

3.

Result:

```
person1 = {
  name: "Rohit",
  age: 20,
  __proto__: Person.prototype
};
```

4.

8.4 The `this` Keyword in Constructor

```
constructor(name, age) {
  this.name = name;  // this points to the new empty object
  this.age = age;
}
```

What `this` means:

- Inside constructor, `this` refers to the newly created empty object
- `this.name = name` creates a `name` property on that object

- Each object gets its own properties

Part 9: Why Methods Go in Prototype

9.1 Properties vs Methods

Properties (name, age):

- Different for each object
- Stored directly in each object
- person1.name ≠ person2.name

Methods (sayHi):

- Same for all objects
- Stored once in prototype
- Shared across all instances

9.2 Visual Comparison

Bad Approach (if methods were in constructor):

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
    this.sayHi = function() {  // BAD! Creates duplicate
      console.log(`Hi ${this.name}`);
    };
  }
}

// Result: Memory waste
person1 = { name: "Rohit", age: 20, sayHi: function(){...} }
person2 = { name: "Mohit", age: 10, sayHi: function(){...} }
person3 = { name: "Mohan", age: 15, sayHi: function(){...} }
// Same function stored 3 times!
```

Good Approach (methods in prototype):

```
class Person {
  constructor(name, age) {
```

```
    this.name = name;
    this.age = age;
  }

  sayHi() {  // Stored once in Person.prototype
    console.log(`Hi ${this.name}`);
  }
}

// Result: Memory efficient
person1 = { name: "Rohit", age: 20 } → Person.prototype.sayHi()
person2 = { name: "Mohit", age: 10 } → Person.prototype.sayHi()
person3 = { name: "Mohan", age: 15 } → Person.prototype.sayHi()
// Same function referenced by all!
```

9.3 Verification

```
console.log(Person.prototype);
// Shows: { sayHi: function() {...} }

console.log(person1.__proto__ === Person.prototype);  // true
console.log(person2.__proto__ === Person.prototype);  // true
```

Part 10: Inheritance with Classes

10.1 The Need for Inheritance

Scenario: Banking Application

```
// Person class (already exists)
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  sayHi() {
    console.log(`Hi ${this.name}`);
  }
}
```

```
// Customer needs name, age + account, balance
// Why rewrite name and age?
```

10.2 Implementing Inheritance

```
class Customer extends Person {
  constructor(name, age, account, balance) {
    super(name, age);  // Call parent constructor
    this.account = account;
    this.balance = balance;
  }

  checkBalance() {
    return this.balance;
  }
}
```

10.3 The `super` Keyword

What `super` does:

```
super(name, age);
```

Equivalent to:

```
// Call Person's constructor with these values
Person.constructor.call(this, name, age);
// This initializes this.name and this.age
```

Why use `super`:

- Customer doesn't own `name` and `age` directly
- They're inherited from Person
- Need to initialize them through parent's constructor

10.4 Complete Inheritance Example

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
```

```
  sayHi() {
    console.log(`Hi ${this.name}`);
  }
}

class Customer extends Person {
  constructor(name, age, account, balance) {
    super(name, age);
    this.account = account;
    this.balance = balance;
  }

  checkBalance() {
    return this.balance;
  }
}

// Create customer
const c1 = new Customer("Mohan", 20, "12", 540);

console.log(c1.name);           // "Mohan" (from Person)
console.log(c1.age);            // 20 (from Person)
console.log(c1.account);        // "12" (own property)
console.log(c1.balance);        // 540 (own property)
c1.sayHi();                     // "Hi Mohan" (from Person)
c1.checkBalance();              // 540 (own method)
```

10.5 Inheritance Chain

```
c1 (Customer instance)
  - account: "12"
  - balance: 540
  - name: "Mohan" (from Person)
  - age: 20 (from Person)
  ↓ c1.__proto__
Customer.prototype
  - checkBalance()
  ↓ Customer.prototype.__proto__
Person.prototype
  - sayHi()
  ↓ Person.prototype.__proto__
Object.prototype
  - hasOwnProperty()
```

```
  - toString()
  ↓ Object.prototype.__proto__
null
```

Part 11: Object.create() Method

11.1 Alternative to Classes

```
const obj = {
  name: "Rohit",
  age: 20
};

// Create new object with obj as prototype
const obj2 = Object.create(obj);
obj2.account = 30;

console.log(obj2.name);     // "Rohit" (from prototype)
console.log(obj2.account);  // 30 (own property)
```

11.2 What Object.create() Does

```
const obj2 = Object.create(obj);
```

Equivalent to:

```
const obj2 = {};
obj2.__proto__ = obj;
```

Result:

- Creates new object
- Sets its prototype to the specified object
- Inherits properties from prototype

Part 12: Important Concepts Summary

12.1 The Prototype Chain Rules

1. **Every object has a prototype** (except null)
2. **Search goes up the chain** until property is found or null is reached
3. **Properties are own**, methods are inherited
4. **Chain ends at null**

12.2 Why Understanding This Matters

Interview Questions Answered:

- Why is `typeof []` equal to "object"? → Arrays inherit from Object.prototype
- Why can functions have properties? → Functions inherit from Object.prototype
- How does inheritance work? → Through the prototype chain

12.3 Best Practices

DO:

- ✅ Use classes for creating multiple similar objects
- ✅ Put methods in class body (they go to prototype)
- ✅ Put unique properties in constructor
- ✅ Use `extends` for inheritance
- ✅ Use `super()` to call parent constructor

DON'T:

- ❌ Manually use `__proto__` in production code
- ❌ Put methods in constructor (memory waste)
- ❌ Create prototype chains more than 2-3 levels deep
- ❌ Modify built-in prototypes (Array.prototype, Object.prototype)

Part 13: Complete Code Examples

13.1 Basic Class

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
```

```
  sayHi() {
    console.log(`Hi ${this.name}`);
  }
}

const p1 = new Person("Rohit", 20);
console.log(p1.name);  // "Rohit"
p1.sayHi();            // "Hi Rohit"
```

13.2 Inheritance

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  sayHi() {
    console.log(`Hi ${this.name}`);
  }
}

class Customer extends Person {
  constructor(name, age, account, balance) {
    super(name, age);
    this.account = account;
    this.balance = balance;
  }

  checkBalance() {
    return this.balance;
  }
}

const c1 = new Customer("Mohan", 20, "12", 540);
console.log(c1);
// Customer {
//   name: "Mohan",
//   age: 20,
//   account: "12",
//   balance: 540
// }
```

```
c1.sayHi();          // "Hi Mohan"
c1.checkBalance();   // 540
```

13.3 Prototype Exploration

```
const arr = [10, 20, 30];

// Level by level
console.log(arr.__proto__);                    // Array.prototype
console.log(arr.__proto__.__proto__);          // Object.prototype
console.log(arr.__proto__.__proto__.__proto__); // null

// Check connections
console.log(arr.__proto__ === Array.prototype);           // true
console.log(Array.prototype.__proto__ === Object.prototype); // true
```

Part 14: Memory Diagram

14.1 Visual Representation

MEMORY LAYOUT:

```
Person class created:
├─ Person.prototype
│   └─ sayHi: function() {...}
│
├─ person1 = new Person("Rohit", 20)
│   ├─ name: "Rohit"
│   ├─ age: 20
│   └─ __proto__ → Person.prototype
│
└─ person2 = new Person("Mohit", 10)
    ├─ name: "Mohit"
    ├─ age: 10
    └─ __proto__ → Person.prototype (same reference!)
```

BENEFITS:
- sayHi() stored once (in Person.prototype)
- Both objects reference the same method
- Memory saved!

Part 15: Key Takeaways

15.1 Core Concepts

1. **Prototypes enable inheritance** in JavaScript
2. **Classes are syntactic sugar** over prototypes
3. **Methods are shared** through prototypes
4. **Properties are unique** to each instance
5. **Everything inherits from Object.prototype** (except null)

15.2 The Big Picture

JavaScript Inheritance = Prototype Chain

object → prototype → prototype → ... → Object.prototype → null

15.3 Why This Matters

- **Memory Efficiency:** Methods stored once
- **Code Reusability:** Inherit from existing classes
- **Understanding JavaScript:** Know how the language really works
- **Debugging:** Understand where methods come from
- **Interviews:** Common topic in technical interviews

Conclusion

Understanding prototypes and classes is fundamental to mastering JavaScript. This knowledge explains:

- Why arrays and functions are objects
- How inheritance works
- Where built-in methods come from
- How to write efficient, maintainable code

The prototype chain is JavaScript's implementation of inheritance, and classes provide a cleaner syntax for working with this system. Master these concepts, and JavaScript will make much more sense!